

FC7xxx UART Integration Manual

Rev.1.0

Revision History

Revision	Date	Changes
1.0	2024/07/17	Initial release

Table of Contents

Chapter 1 Introduction	4
1.1 Introduction	4
Chapter 2 Building	5
2.1 Dependencies on Other Modules	5
2.2 Files Required for Compile	5
2.3 Add Plug-ins	6
Chapter 3 Memory	7
3.1 Sections in Memory Map	7
Chapter 4 Exclusive area.....	9
Chapter 5 Error Report	11
5.1 Det	11
Chapter 6 Function Calls.....	13
6.1 Function Calls during Startup	13
6.2 Function Calls during Shutdown.....	13
6.3 Function Calls during Wake-up.....	13
Chapter 7 Other Requirements.....	14
7.1 Notification, Callback, Callout	14
Macros	14
Chapter 8 Integration Steps.....	15

Chapter 1 Introduction

1.1 Introduction

This integration manual describes the integration requirements for the UART module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Mcu: This module provides the clock reference point for UART module
- Port: This module provides the port configurations if want output UART signal through pin.
- Common: This module is the basic module which used to choose the chip.
- EcuM: is required for selecting the reference to the wakeup source for every UART controller.
- Det: is required for signaling the default error detection (parameters out of range, null pointers, etc)
- Dem: is required for signaling the production error detection (hardware failure, etc).

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for tracing Development errors.
- Dem: This module is necessary for enabling Development error detection.
- EcuM: This module provides the EcuM callback functions when wakeup support is enabled.

Module initialization dependency

- Mcu: To use the UART module, the user needs to initialize the MCU module first
- Port: To output UART signals, the user needs to initialize PORT module first and assign the UART pins.
- DMA: To use UART module DMA function, the user needs to initialize DMA module first.

2.2 Files Required for Compile

Uart module files:

- _MCAL/Src/Uart/Src/CDD_Uart.c
- _MCAL/Src/Uart/Src/LLD_Uart.c
- _MCAL/Src/Uart/Src/CDD_Uart_Irq.c
- _MCAL/Src/Uart/Src/CDD_Uart.h
- _MCAL/Src/Uart/Src/LLD_Uart.h
- _MCAL/Src/Uart/Src/Uart_GeneralTypes.h
- _MCAL/Src/Uart/Src/Uart_Irq.h
- _MCAL/Src/Uart/Src/Uart_MemMap.h
- _MCAL/Src/Uart/Src/Uart_Reg.h
- _MCAL/Src/Uart/Src/Uart_RegOps.h
- _MCAL/Src/Uart/Src/Uart_Types.h
- _MCAL/Src/Uart/Src/Uart_Version.h
- _MCAL_generate/src/CDD_Uart_PBcfg.c
- _MCAL_generate/src/CDD_Uart_PBcfg.h
- _MCAL_generate/include/CDD_Uart_Cfg.h
- _MCAL_generate/include/CDD_Uart_Defines.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h

- _MCAL/Src/Common/include/Std_Types.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h

Det module files:

- Det.h

Rte module files:

- SchM_Uart.h

Dem module files:

- Dem.h

EcuM module files:

- EcuM_Cbk.h

2.3 Add Plug-ins

UART module plug-ins are developed for EB tresos Studio, so, to use UART plug-ins on the EB tresos Studio, the user needs to add the UART module to the EB plug-ins folder first.

- 1) Copy the UART module(_MCAL/EB_Plugins/eclipse/plugins/Uart) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
- 2) Set the UART module output location folder for the generated source file and header files.
- 3) Use EB tresos to configure the UART module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
UART_START_SEC_CONFIG_DATA_8 UART_STOP_SEC_CONFIG_DATA_8 UART_START_SEC_CONFIG_DATA_16 UART_STOP_SEC_CONFIG_DATA_16 UART_START_SEC_CONFIG_DATA_32 UART_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).
UART_START_SEC_CONFIG_DATA_UNSPECIFIED UART_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
UART_START_SEC_CONST_BOOLEAN UART_STOP_SEC_CONST_BOOLEAN UART_START_SEC_CONST_8 UART_STOP_SEC_CONST_8 UART_START_SEC_CONST_16 UART_STOP_SEC_CONST_16 UART_START_SEC_CONST_32 UART_STOP_SEC_CONST_32 UART_START_SEC_CONST_UNSPECIFIED UART_STOP_SEC_CONST_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are read only (rodata).
UART_START_SEC_CODE UART_STOP_SEC_CODE UART_START_SEC_RAMCODE UART_STOP_SEC_RAMCODE UART_START_SEC_CODE_AC UART_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code (text).
UART_START_SEC_VAR_NO_INIT_BOOLEAN UART_STOP_SEC_VAR_NO_INIT_BOOLEAN UART_START_SEC_VAR_NO_INIT_8 UART_STOP_SEC_VAR_NO_INIT_8 UART_START_SEC_VAR_NO_INIT_16 UART_STOP_SEC_VAR_NO_INIT_16 UART_START_SEC_VAR_NO_INIT_32 UART_STOP_SEC_VAR_NO_INIT_32 UART_START_SEC_VAR_NO_INIT_UNSPECIFIED UART_STOP_SEC_VAR_NO_INIT_UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
UART_START_SEC_VAR_INIT_BOOLEAN UART_STOP_SEC_VAR_INIT_BOOLEAN UART_START_SEC_VAR_INIT_8 UART_STOP_SEC_VAR_INIT_8 UART_START_SEC_VAR_INIT_16 UART_STOP_SEC_VAR_INIT_16 UART_START_SEC_VAR_INIT_32 UART_STOP_SEC_VAR_INIT_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).

UART_START_SEC_VAR_INIT_UNSPECIFIED		
UART_STOP_SEC_VAR_INIT_UNSPECIFIED		

Chapter 4 Exclusive area

UART module using the services of Schedule Manger (SchM) for entering and exiting critical regions.

The following critical regions are used in the UART driver:

- LLD_Uart.c:
 - UART_EXCLUSIVE_AREA_00 is used in Uart_LLD_SyncSend
 - UART_EXCLUSIVE_AREA_01 is used in Uart_LLD_AsyncSend.
 - UART_EXCLUSIVE_AREA_02 is used in Uart_LLD_SyncReceive.
 - UART_EXCLUSIVE_AREA_03 is used in Uart_LLD_AsyncReceive.
 - UART_EXCLUSIVE_AREA_04 is used in Uart_LLD_GetReceiveStatus.
 - UART_EXCLUSIVE_AREA_05 is used in Uart_LLD_GetTransmitStatus.

Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
FCUART0	FCUART0_RxTx_IRQHandler	100
FCUART1	FCUART1_RxTx_IRQHandler	101
FCUART2	FCUART2_RxTx_IRQHandler	102
FCUART3	FCUART3_RxTx_IRQHandler	103
FCUART4	FCUART4_RxTx_IRQHandler	104
FCUART5	FCUART5_RxTx_IRQHandler	105
FCUART6	FCUART6_RxTx_IRQHandler	106
FCUART7	FCUART7_RxTx_IRQHandler	107
FCUART8	FCUART8_RxTx_IRQHandler	108
FCUART9	FCUART9_RxTx_IRQHandler	109
FCUART10	FCUART10_RxTx_IRQHandler	110
FCUART11	FCUART11_RxTx_IRQHandler	111
FCUART12	FCUART12_RxTx_IRQHandler	112
FCUART13	FCUART13_RxTx_IRQHandler	113
FCUART14	FCUART14_RxTx_IRQHandler	114
FCUART15	FCUART15_RxTx_IRQHandler	115

Chapter 5 Error Report

5.1 Det

Function Name	Error Type
Uart_Init	UART_E_ALREADY_INITIALIZED; UART_E_INIT_FAILED; UART_E_PARAM_CONFIG
Uart_Deinit	UART_E_UNINIT; UART_E_DEINIT_FAILED;
Uart_SetBaudrate	UART_E_PARAM_CONFIG; UART_E_UNINIT; UART_E_INVALID_PARAMETER; UART_E_INVALID_CHANNEL; UART_E_PARAM_CONFIG; UART_E_INVALID_CHANNEL; UART_E_CHANNEL_BUSY
Uart_GetBaudrate	UART_E_PARAM_CONFIG; UART_E_UNINIT; UART_E_INVALID_CHANNEL; UART_E_INVALID_POINTER;
Uart_AbortTransfer	UART_E_PARAM_CONFIG; UART_E_UNINIT; UART_E_INVALID_CHANNEL;
Uart_SetTransBuffer	UART_E_PARAM_CONFIG; UART_E_UNINIT; UART_E_INVALID_CHANNEL; UART_E_INVALID_POINTER; UART_E_INVALID_PARAMETER;
Uart_SyncSend	UART_E_PARAM_CONFIG; UART_E_UNINIT; UART_E_INVALID_CHANNEL; UART_E_INVALID_POINTER; UART_E_INVALID_PARAMETER; UART_E_CHANNEL_BUSY; UART_E_TIMEOUT;
Uart_SyncReceive	UART_E_PARAM_CONFIG; UART_E_UNINIT; UART_E_INVALID_CHANNEL; UART_E_INVALID_POINTER; UART_E_INVALID_PARAMETER; UART_E_CHANNEL_BUSY; UART_E_TIMEOUT;
Uart_AsyncReceive	UART_E_PARAM_CONFIG; UART_E_UNINIT; UART_E_INVALID_CHANNEL; UART_E_INVALID_POINTER;

	UART_E_INVALID_PARAMETER; UART_E_CHANNEL_BUSY;
Uart_AsyncSend	UART_E_PARAM_CONFIG; UART_E_UNINIT; UART_E_INVALID_CHANNEL; UART_E_INVALID_POINTER; UART_E_INVALID_PARAMETER; UART_E_CHANNEL_BUSY;
Uart_GetStatus	UART_E_PARAM_CONFIG; UART_E_UNINIT; UART_E_INVALID_CHANNEL; UART_E_INVALID_POINTER;
Uart_GetVersionInfo	UART_E_INVALID_POINTER

Chapter 6 Function Calls

6.1 Function Calls during Startup

Before using the Uart peripheral, the Uart driver must be initialized.

During STARTUP, the API called to initialize the driver is Uart_Init.

After the Uart module is initialized, each Uart channel must also be initialized before using it. This is also done by the Uart_Init service.

Before initializing the Uart driver, the MCU module should be initialized.

The PORT, MCU and DMA modules (if using the DMA option) should be initialized before the Uart driver.

6.2 Function Calls during Shutdown

The Uart module can be deinitialized by calling Uart_Delnit(). It must be ensured that all hardware channels are not in the process of transmission during deinitialization. Otherwise, the Delnit function will not be completed successfully and the driver will report a channel error if the user enables Det error in the EB configuration.

6.3 Function Calls during Wake-up

N/A

Chapter 7 Other Requirements

7.1 Notification, Callback, Callout

The driver provides callback notifications for asynchronous transfers. The driver gives possibility to configure callbacks for each channel to be called at the end of a transmission.

- **Receive callback:** In interrupt mode, the callback is called twice. The first time is when the Rx buffer is full, giving the user a chance to call `Uart_SetTransBuffer()` to provide a new buffer for continued reception. If a new buffer is not provided in the first call, a second call is made to end reception. The event parameter in the callback signature is used to distinguish between the two calls. For the first call, the parameter is `UART_EVENT_RX_FULL`, while for the second call, the callback parameter is `UART_EVENT_END_TRANSFER`.
- **Transmit callback:** In interrupt mode, the callback is also called twice. In the first call, the event parameter is `UART_EVENT_TX_EMPTY`, indicating that all bytes in the provided buffer have been transmitted. In this call, `Uart_SetTransBuffer()` can be called to change the buffer and transmit more data. If the transmission is finished (all data has been transmitted), a second call is made with the `UART_EVENT_END_TRANSFER` parameter.
- **Callbacks on errors:** The driver handles the following errors: rx overrun, noise errors, framing errors, and parity errors. The callback is called with the `UART_EVENT_ERROR` argument.

Macros

Please check various definitions available in Common module's include file `Mcal.h` for details.

- If OS is not used:
`AUTOSAR_OS_NOT_USED` needs to be defined.

In this case, If `USE_SW_VECTOR_MODE` is not defined, the user needs to map the interrupt vector table function to the ISR function, for example:

```
extern ISR(UART_0_IRQHandler);

UART_TEXT_SECTION ISR(UART_0_IRQHandler)
{
    Uart_LLD_IrqHandler(0U);
    EXIT_INTERRUPT();
}
```

- If OS is used:
Do not define `AUTOSAR_OS_NOT_USED`.

Chapter 8 Integration Steps

- 1) Configure the UART module and generate configuration files (please refer to Building chapter for details).
- 2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
- 3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
- 4) Build the UART module with dependent modules.